



OEMiROT for Nucleo-H553ZG

Notes about the modified example

2026 Aug 13

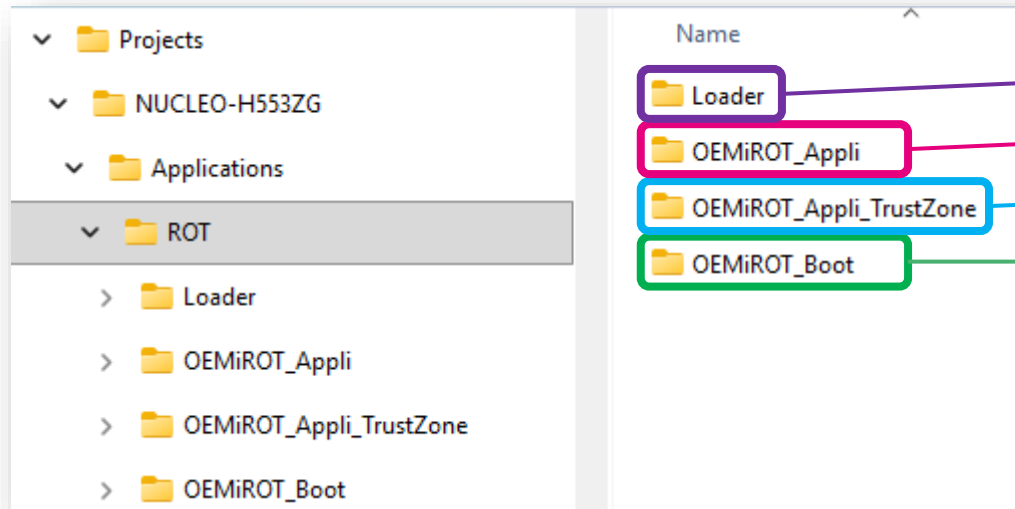
- 1 Overview
- 2 Memory layout (primary only config)
- 3 Features added/modified
- 4 How to use the example

- Where to get the modified OEMiROT (including examples)
 - Internal:
 - Sharepoint folder: [v1.7.0 oemirot invt](#)
 - Two versions available as of now
 - [h553 v1.7.0 oemirot invt primaryonly sysbl.7z](#)
 - [h553 v1.7.0 oemirot invt primaryonly.7z](#)
 - External:
 - Github: <https://github.com/STChens/STM32CubeH5> (branch v1.7.0_oemirot_invt)
- Target HW board: Nucleo-H553ZG
- Base version of CubeFW package: STM32CubeH5 v1.7.0
- Supported IDE: MDK-ARM (tested with µVision v5.43.1.0)



Content of the Package

- Once unzip the package, you will see two folders

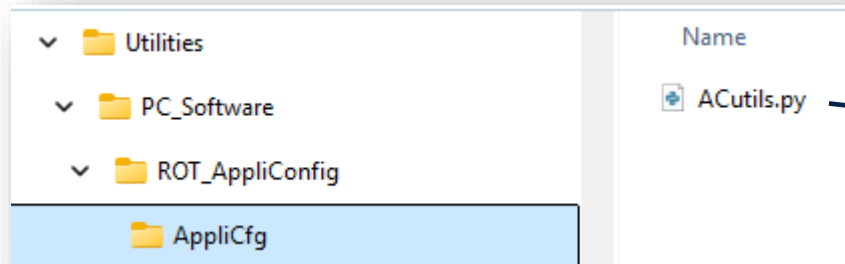


Example user loader code (to download new version app image)

Example Application without isolation (TZ Secure only App)

Example Application with isolation (TZ S+NS App)

OEMiROT boot code with modifications



Modified AppliCfg python code (to facilitate OEMiROT Boot postbuild script)
See more details in the notes about OEMiROT Boot postbuild

Memory Layout (for Primary Only config)

Memory Layout of Primary Only config

- Flash layout according to default config in flash_layout.h
 - Secure Only App (FLASH_NS_PARTITION_SIZE = 0)
 - without** system bootloader feature

Area purpose	AREA ID	Offset	Start	End	size (B hex)	size (KB)	Nb pages	WRP(OB) protection	HDP (OB) protection	Flash SECWM (OB)
OEMiROT	BL2	0	C000000	C00FFFF	20000	64	8	Y	Y	S
LOADER		10000	C010000	C01FFFF	10000	64	8	Y	N	S
SCRATCH area	SCRTACH	NA	NA	NA	NA	NA	NA	NA	NA	NA
Data Active	4	NA	NA	NA	NA	NA	NA	NA	NA	NA
App Active	0	20000	C020000	C0FBFFF	DC000	880	110	N	N	S
App Download	2	NA	NA	NA	NA	NA	NA	NA	NA	NA
Data Download	6	NA	NA	NA	NA	NA	NA	NA	NA	NA
Free space (can be used as NVM area for app)	Not defined	FC000	80FC000	80FFFFFF	4000	16	2	N	N	NS

Memory Layout of Primary Only config

- Flash layout according to default config in flash_layout.h
 - Secure Only App (`FLASH_NS_PARTITION_SIZE = 0`)
 - with** system bootloader feature

Area purpose	AREA ID	Offset	Start	End	size (B hex)	size (KB)	Nb pages	WRP(OB) protection	HDP (OB) protection	Flash SECWM (OB)	Flash SECBB (config by OEMiROT)
OEMiROT	BL2	0	C000000	C00FFFF	20000	64	8	Y	Y	S	S
LOADER		10000	C010000	C01FFFF	10000	64	8	Y	N	S	S
SCRATCH area	SCRTACH	NA	NA	NA	NA	NA	NA	NA	NA	NA	
Data Active	4	NA	NA	NA	NA	NA	NA	NA	NA	NA	
App Active	0	20000	C020000	C0FBFFF	DC000	880	110	N	N	NS	S
App Download	2	NA	NA	NA	NA	NA	NA	NA	NA	NA	
Data Download	6	NA	NA	NA	NA	NA	NA	NA	NA	NA	
Free space (can be used as NVM area for app)	Not defined	FC000	80FC000	80FFFFFF	4000	16	2	N	N	NS	NS

Memory Layout of Primary Only config

- Flash layout according to default config in flash_layout.h
 - TZ S/NS App (merge into 1 image) (`FLASH_NS_PARTITION_SIZE` $\neq$ 0)
 - without** system bootloader feature

Area purpose	AREA ID	Offset	Start @	End @	size (B hex)	size (KB)	Nb pages	WRP(OB) protection	HDP (OB) protection	Flash SECWM (OB)
OEMiROT	BL2	0	C000000	C00FFFF	20000	64	8	Y	Y	S
LOADER		10000	C010000	C01FFFF	10000	64	8	Y	N	S
Active App S + App NS (merge to 1 image)	0	20000 (S App)	C020000	C025FFF	6000	24	3	N	N	S
		26000 (NS App)	8026000	8075FFF	50000	320	40	N	N	NS
Free space (can be used as NVM area for app)	Not defined	76000	8076000	80FFFFFF	4000	552	69	N	N	NS

Memory Layout of Primary Only config

- Flash layout according to default config in flash_layout.h
 - TZ S/NS App (merge into 1 image) (`FLASH_NS_PARTITION_SIZE` $\neq$ 0)
 - with system bootloader feature

Area purpose	AREA ID	Offset	Start @	End @	size (B hex)	size (KB)	Nb pages	WRP(OB) protection	HDP (OB) protection	Flash SECWM (OB)	Flash SECBB (config by OEMiROT)
OEMiROT	BL2	0	C000000	C00FFFF	20000	64	8	Y	Y	S	S
LOADER		10000	C010000	C01FFFF	10000	64	8	Y	N	S	S
Active App S + App NS (merge to 1 image)	0	20000 (S App)	C020000	C025FFF	6000	24	3	N	N	NS	S
		26000 (NS App)	8026000	8075FFF	50000	320	40	N	N	NS	NS
Free space (can be used as NVM area for app)	Not defined	76000	8076000	80FFFFFF	4000	552	69	N	N	NS	NS

Features Added/Modified

Features added vs. v1.7.0 base version

- OEMiROT Boot

- Added support of **PRIMARY ONLY** config for memory layout
 - Macro **MCUBOOT_PRIMARY_ONLY** is by default enabled in
 - `Projects\NUCLEO-H553ZG\Applications\ROT\OEMiROT_Boot\Inc\flash_layout.h`
- In case of no valid image in primary slot or user button is pressed during boot, OEMiROT will jump to **user loader** instead of system bootloader
- WA to deal with the case that .c/.h files are encrypted on user host PC
 - Normally the postbuild script of OEMiROT BOOT project will modify some application .h files (e.g. `main.h`, `appli_flash_layout.h`) according to memory layout definition
 - If those .h files are encrypted on user host PC, the script will fail when trying to modify those .h files
 - WA is to have a copy of those .h files as .h.txt files
 - Postbuild script will try to modify .h.txt files instead and then copy the modified .h.txt files to the target .h files
 - If the .h.txt files does not exist, the postbuild script will still try to modify .h files



Features added vs. v1.7.0 base version

- User Loader example
 - Added example loader for new version FW download
 - Example loader added in folder `Projects\NUCLEO-H553ZG\Applications\ROT\Loader`
 - User loader will be considered as part of BL2 (Immutable RoT) in the memory layout
 - So, User Loader will be write protected (covered by WRP group OB settings)
 - That means user loader code cannot be modified through FW update
 - But user loader code area will not be HDP protected because we allow application to jump to user loader
 - That means HDP L1 protection only covers OEMiROT BOOT area
 - Features of the example user loader code includes
 - New image download from UART with Ymodem protocol
 - Jumping to system bootloader
 - This feature is controlled by definition of macro **USE_SYSTEM_LOADER** in `Projects\NUCLEO-H553ZG\Applications\ROT\OEMiROT_Boot\Inc\flash_layout.h`
 - Comment out “`#define USE_SYSTEM_LOADER`” in `flash_layout.h` if such feature is not needed
 - USB CDC test demo



Features added vs. v1.7.0 base version

- Application Example
 - Added Example of Application without isolation (TZ enabled secure only app)
 - Example app added in folder `Projects\NUCLEO-H553ZG\Applications\ROT\OEMiROT_Appli`
 - Added the function of jumping from Application to **User Loader**
 - For **Primary Only** configuration, jumping from application to **system bootloader** is also allowed in the example
 - But this feature has some security concerns, that user must be aware of
 - Please see more details in the next pages about this
 - This feature is controlled by a macro **USE_SYSTEM_LOADER** in `appli_flash_layout.h`
 - Undefine this macro to disable the feature of jumping from application to system bootloader if it is not needed

About Using System Bootloader in Primary Only Config

- System boot loader is running in NS state, so it can only access NS flash pages
- In case of “Primary Only” config, if we allow system bootloader to access primary slot of flash area, we have to make sure those pages are in NS state before jumping to system bootloader
- In default implementation of OEMiROT, FLASH SECWM OB will be set according to the total area covered by both OEMiROT code and Application S code
 - In such case, even if system bootloader can run, it cannot read/write to the secure area of application primary slot
- An WA is implemented in the modified OEMiROT
 - In provisioning step, FLASH SECWM OB will only cover OEMiROT + loader code area
 - During OEMiROT execution, it will configure FLASH SECBB to make the flash pages used by Application primary slot (S app part) become **secure**
 - This enables the possibility to allow secure code (e.g. user loader) to re-configure FLASH SECBB registers to make Application primary slot area become NS again before jumping to system bootloader. Then system bootloader will have full R/W access to App primary slot flash area



About Using System Bootloader in Primary Only Config

- In case of jumping from **user loader** to system bootloader
 - Before launching system bootloader, User loader will
 - Set FLASH SECBB to make flash pages of App S area become NS
 - This is to enable full access of Primary slot by system bootloader
 - Erase the content of flash pages of App S area
 - This is to avoid exposing Application code (in clear) from primary slot

Static config in OB

Flash Area	Flash SECWM (OB)
OEMiROT	S
LOADER	S
App Active	NS

OEMiROT → Application

Flash Area	Flash SECWM (OB)	Flash SECBB
OEMiROT	S	S
LOADER	S	S
App Active	NS	S

Loader Execution → system bootloader

Flash Area	Flash SECWM (OB)	Flash SECBB (loader running)	Flash SECBB (before loader jump to system bl)
OEMiROT	S	S	S
LOADER	S	S	S
App Active	NS	S	NS

About Using System Bootloader in Primary Only Config

- In case of jumping from **Application** to system bootloader
 - Similar thing of re-configuring FLASH SECBB must be included in App in order to allow system bootloader to have full access to primary slot of App flash area for Primary only config
 - One thing that is different vs. jumping from User Loader to system bootloader is that, Application itself has secure part and if Application set FLASH SECBB to NS for App flash pages, the S code from flash cannot execute
 - The way to WA this is to let this part of code of App to run in RAM
 - Special care must be taken in linker file. E.g. linker file of secure app (TZ S+NS project)
 - ro part of secure_nsc.o will be placed in RAM and only SRAM2 is used (because SRAM1 and SRAM3 will be configured as NS before launching system bootloader)

```
27 #define S_DATA_START 0x30020000
28 #define S_DATA_SIZE 0xFFFF
29
30 #define S_CODE_LIMIT (S_CODE_START + S_APP_CODE_SIZE - 1)
31
32 #define CMSE_VENEER_REGION_START S_CODE_LIMIT + 1
33 #define CMSE_VENEER_REGION_SIZE 0x400 /* Include TLVs and trailer. User configurable */
34
35 LR_CODE S_CODE_START {
36     ER_CODE S_CODE_START S_APP_CODE_SIZE {
37         *.o (RESET +First)
38         .ANY (+RO)
39     }
40
41     ER_DATA S_DATA_START S_DATA_SIZE {
42         secure_nsc.o (+RO)
43         .ANY (+ZI +RW)
44     }
45 }
```


About Using System Bootloader in Primary Only Config

- In case of jumping from **Application** to system bootloader
 - Security concerns of such example that user must be aware of
 - In the current example, before Application jumping to system bootloader, the flash content of primary slot is not erased
 - That means if system bootloader starts, it is possible to connect to the board through communication port (e.g. usb) using STM32CubeProgrammer and application code from primary slot will be visible
 - Recommendation is that flash content of primary slot shall be removed before launching system bootloader
 - To achieve that, more driver code need to be placed in RAM (e.g. flash driver) in secure App project

How to use the example

Get the package

- If using code from github directly
 - Get the full package from <https://github.com/STChens/STM32CubeH5> (branch v1.7.0_oemirot_invt)
- Otherwise
 - Get the base version of STM32H5 Cube package v1.7.0
 - Get the zip package of modified OEMiROT from sharepoint
 - Unzip the package and copy to the v1.7.0 base package folder (replace the files in the destination)



Selection of Application w/ or wo/ Isolation

- Config to use Application with or without Isolation

- Projects\NUCLEO-H553ZG\ROT_Provisioning\env.bat

- To use OEMiROT_Appli_TrustZone

```
24 ::set oemirot_appli_path_project=Applications/ROT/OEMiROT_Appli
25 ::set oemirot_appli_path_project=Templates/ROT/OEMiROT_Appli_TrustZone
26 set oemirot_appli_path_project=Applications/ROT/OEMiROT_Appli_TrustZone
```

- To use OEMiROT_Appli

```
24 set oemirot_appli_path_project=Applications/ROT/OEMiROT_Appli
25 ::set oemirot_appli_path_project=Templates/ROT/OEMiROT_Appli_TrustZone
26 ::set oemirot_appli_path_project=Applications/ROT/OEMiROT_Appli_TrustZone
```

- Projects\NUCLEO-H553ZG\Applications\ROT\OEMiROT_Boot\Inc\flash_layout.h

- To use OEMiROT_Appli_TrustZone: FLASH_NS_PARTITION_SIZE **MUST NOT BE 0**

```
184 /* BL2 partitions size */
185 #define FLASH_NS_PARTITION_SIZE (0x50000) /* 320 KB for NS partition (except for FULL SECURE) */
186 // #define FLASH_NS_PARTITION_SIZE (0) /* 0 for FULL SECURE */
```

- To use OEMiROT_Appli: FLASH_NS_PARTITION_SIZE **MUST BE 0**

```
184 /* BL2 partitions size */
185 // #define FLASH_NS_PARTITION_SIZE (0x50000) /* 320 KB for NS partition (except for FULL SECURE) */
186 #define FLASH_NS_PARTITION_SIZE (0) /* 0 for FULL SECURE */
```



Build the projects

- Project Build order
 - OEMiROT_Boot project must be built first
 - That's because the postbuild script will modify some files of application (e.g. linker file, header files)
 - Then Loader or Application can be built
 - There is no special order to be followed for this
 - Whether to build OEMiROT_Appli_TrustZone or OEMiROT_Appli project, depends on the Application config selection (as described in the previous page)
 - For TZ Application with Isolation, Secure App must be built before Non-Secure application

Files generated after build steps

- OEMiROT_BOOT\Binary\Boot.bin:
 - OEMiROT Boot binary
- Loader\Binary\Loader.bin:
 - User loader binary
- OEMiROT_BOOT\Binary\OEMiROT_Boot.bin:
 - binary that combines OEMiROT Boot binary and loader binary → This is the file to be programmed by the provisioning script
 - The postbuild script of loader project and OEMiROT Boot project will both do the merging of the two binaries as long as the file exists

Files generated after build steps (Primary only config)

- OEMiROT_Appli\Binary
 - rot_tz_s_app.bin
 - Original App binary (Secure Only) to be signed
 - rot_tz_s_app_enc_sign.bin
 - Encrypted and signed binary for fw update
 - To be used for user loader download through UART Ymodem
 - rot_tz_s_app_enc_sign.hex
 - Encrypted and signed binary for fw update
 - To be used for download through CubeProgrammer with system bootloader
 - rot_tz_s_app_init_sign.hex
 - Signed image for programming by provisioning script
- OEMiROT_Appli_TrustZone\Binary
 - rot_tz_app.bin
 - Original App binary (S+NS merged into one binary) to be signed
 - rot_tz_app_enc_sign.bin
 - Encrypted and signed binary for fw update
 - To be used for user loader download through UART Ymodem
 - rot_tz_app_enc_sign.hex
 - Encrypted and signed binary for fw update
 - To be used for download through CubeProgrammer with system bootloader
 - rot_tz_app_init_sign.hex
 - Signed image for programming by provisioning script

Provisioning

- To provision the images run the provisioning script as usual and follow the steps as usual
 - `Projects\NUCLEO-H553ZG\ROT_Provisioning\OEMiROT\provisioning.bat`
- If “OPEN” is selected as the targeted final product state (e.g. in dev phase)
 - In case you want to reprogram the binaries (OEMiROT, Loader or Application), you can just re-run the script `ob_flash_programming.bat` (which is in the same folder as `provisioning.bat`)
- Otherwise, if you want to redo the provisioning, please run DA procedure to do a full regression first

Run the example: Launch Application

- After provisioning, connect a com terminal on host PC
- Power on or reset the board
- Then you can see some log similar to the one shown on as follows

```
[INF] BOOT HAL built on: Aug 12 2026 16:07:44
[INF] TAMPER Activated
[INF] Flash operation: Op=0x0, Area=0x0, Address=0x0
[INF] BL2 start: 0c000000
[INF] BL2 size : 00010000
[INF] Loader start: 0c010000
[INF] Loader size : 00010000
[INF] Flash area 0 offset: 00020000
[INF] Flash area 0 size : 00056000
[INF] HDP end: 0000ffff
[INF] Starting bootloader OEMiROT
[WRN] Image 0 is installed
[INF] Starting validation of primary slot(s)
[INF] verify counter 0 1000000 0
[INF] counter 0 : ok
[INF] verify sig key id 0
[INF] checking public key 47 5b
[INF] verifying signature hlen 20
[INF] signature OK
[INF] Bootloader chainload address offset: 0x20000
[INF] Jumping to the first image slot
```

```
=====
=                (C) COPYRIGHT 2023 STMicroelectronics                =
=                Built on Aug 12 2026 16:12:25                        =
=                User App #A                                          =
=====
```

```
===== Main Menu =====
```

```
Start User Loader ----- 0
```

```
Start System Bootloader ----- 1
```

```
Selection :
```



Run the example: Launch User Loader

- If user button is pressed during power on/reset
 - User loader will be launched
- It is also possible to jump to user loader from Application menu
 - By selecting option '0' in App menu

```
[INF] BOOT HAL built on: Aug 12 2026 16:07:44
[INF] TAMPER Activated
[INF] Flash operation: Op=0x0, Area=0x0, Address=0x0
[INF] BL2 start: 0c000000
[INF] BL2 size : 00010000
[INF] Loader start: 0c010000
[INF] Loader size : 00010000
[INF] Flash area 0 offset: 00020000
[INF] Flash area 0 size : 00056000
[INF] HDP end: 0000ffff
[INF] Jumping to UART YModem bootloader @0c010000
```

```
=====
=                (C) COPYRIGHT 2026 STMicroelectronics                =
=                Build time: Aug 12 2026 16:13:13                    =
=                LOCAL LOADER                                        =
=====
```

```
===== New Fw Image =====
```

```
Jump to system bootloader ----- b
Reset to trigger Installation ----- 1
Download App Image ----- 2
Exit from FW update menu ----- x
```

```
=====
=                (C) COPYRIGHT 2023 STMicroelectronics                =
=                Built on Aug 13 2026 11:21:12                        =
=                User App #A                                          =
=====
```

```
===== Main Menu =====
```

```
Start User Loader ----- 0
Start System BootLoader ----- 1
Selection :
```

```
=====
=                (C) COPYRIGHT 2026 STMicroelectronics                =
=                Build time: Aug 12 2026 16:13:13                    =
=                LOCAL LOADER                                        =
=====
```

```
===== New Fw Image =====
```

```
Jump to system bootloader ----- b
Reset to trigger Installation ----- 1
Download App Image ----- 2
Exit from FW update menu ----- x
```

Run the example: User Loader menu

- If user button is pressed during power on/reset
 - User loader will be launched

```
[INF] BOOT HAL built on: Aug 12 2026 16:07:44
[INF] TAMPER Activated
[INF] Flash operation: Op=0x0, Area=0x0, Address=0x0
[INF] BL2 start: 0c000000
[INF] BL2 size : 00010000
[INF] Loader start: 0c010000
[INF] Loader size : 00010000
[INF] Flash area 0 offset: 00020000
[INF] Flash area 0 size : 00056000
[INF] HDP end: 0000ffff
[INF] Jumping to UART YModem bootloader @0c010000
```

```
=====
=                (C) COPYRIGHT 2026 STMicroelectronics                =
=                Build time: Aug 12 2026 16:13:13                      =
=                LOCAL LOADER                                         =
=====
```

```
===== New Fw Image =====
```

```
Jump to system bootloader ----- b
Reset to trigger Installation ----- 1
Download App Image ----- 2
Exit from FW update menu ----- x
```

- In user loader menu you can
 - Select 'b' to start system bootloader
 - Or select '2' to start application image downloading
 - Or select '1' to reset the system
 - This is usually used after new image fw downloading using menu item '2'
 - Or select 'x' to exit from loader menu
 - In this case, USB CDC test demo will be started

Run the example: Start System Bootloader

- Start system bootloader from User Loader menu by selecting option 'b'

- Note for Primary only config

- Once 'b' is typed in User Loader menu

- Primary slot content will be erased
 - This is for security considerations
 - Because application code in primary slot is in clear
 - Once system bootloader is running, it is possible to connect to the board using CubeProgrammer through communication ports (e.g. UART, USB)
 - To avoid disclosing the Application code content in such case, the content of primary slot will be erased before User Loader jumping to system bootloader
- Protection will be reconfigured to allow system bootloader to access primary slot
 - Special thing is that FLASH SECBB will be configured to change App primary slot to all NS state.
 - This is to allow system bootloader (which runs in NS mode) to write to primary slot (which contains secure pages at run time)

```
=====
=                (C) COPYRIGHT 2026 STMicroelectronics                =
=                Build time: Aug 12 2026 16:13:13                    =
=                LOCAL LOADER                                       =
=====

===== New Fw Image =====

Jump to system bootloader ----- b
Reset to trigger Installation ----- 1
Download App Image ----- 2
Exit from FW update menu ----- x

Erase download area before jumping to the bootloader -- Erasing download area

Start config before jumping to the bootloader
Standard Bootloader started
If you want to connect through USART interface, disconnect your TeraTerm
Start download with STM32CubeProgrammer through supported interfaces
(USART/SPI/I2C/USB)
```



Run the example: Download App Image in CubeProg

- Once system bootloader is launched, it is possible to connect to the board via certain port (e.g. usb) using STM32CubeProgrammer and download new version fw image (xxx_enc_sign.hex)

The screenshot displays the STM32CubeProgrammer software interface. The top bar indicates 'Memory & File editing' and 'Connected' status. The left sidebar contains icons for editing, downloading, and other functions. The main window is divided into two panes. The left pane, titled 'Device memory', shows a file named 'rot_tz_app_enc_sign.hex' loaded. It includes search fields for address (0x08020), data (0x800), and a 'Find Data' button. Below these is a table of memory addresses and their corresponding data.

Address	0	4	8	C	ASCII
0x08020030	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF	yyyyyyyyyyyyyyyy
0x08020040	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF	yyyyyyyyyyyyyyyy
0x08020050	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF	yyyyyyyyyyyyyyyy
0x08020060	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF	yyyyyyyyyyyyyyyy
0x08020070	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF	yyyyyyyyyyyyyyyy
0x08020080	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF	yyyyyyyyyyyyyyyy
0x08020090	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF	yyyyyyyyyyyyyyyy
0x080200A0	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF	yyyyyyyyyyyyyyyy
0x080200B0	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF	yyyyyyyyyyyyyyyy

The right pane, titled 'USB configuration', shows the current USB settings. It includes a 'USB' dropdown menu, a 'Disconnect' button, and a 'USB configuration' section with the following details:

- Port: USB1
- Serial number: 387633693235
- PID: 0xDF11
- VID: 0x0483
- Timeout (ms): 30000
- Read Unprotect (MCU): ☐
- TZEN Regression (MCU): ☐

- After the download, reboot the board and new version app will be installed and run

Run the example: Download App image in User Loder

- In User Loader menu, you can start App image download with option '2'
- Send the app fw update binary image in Tera Term with Ymodem

```
=====
=              (C) COPYRIGHT 2026 STMicroelectronics              =
=              Build time: Aug 12 2026 16:13:13                    =
=              LOCAL LOADER                                       =
=====

===== New Fw Image =====

Jump to system bootloader ----- b

Reset to trigger Installation ----- 1

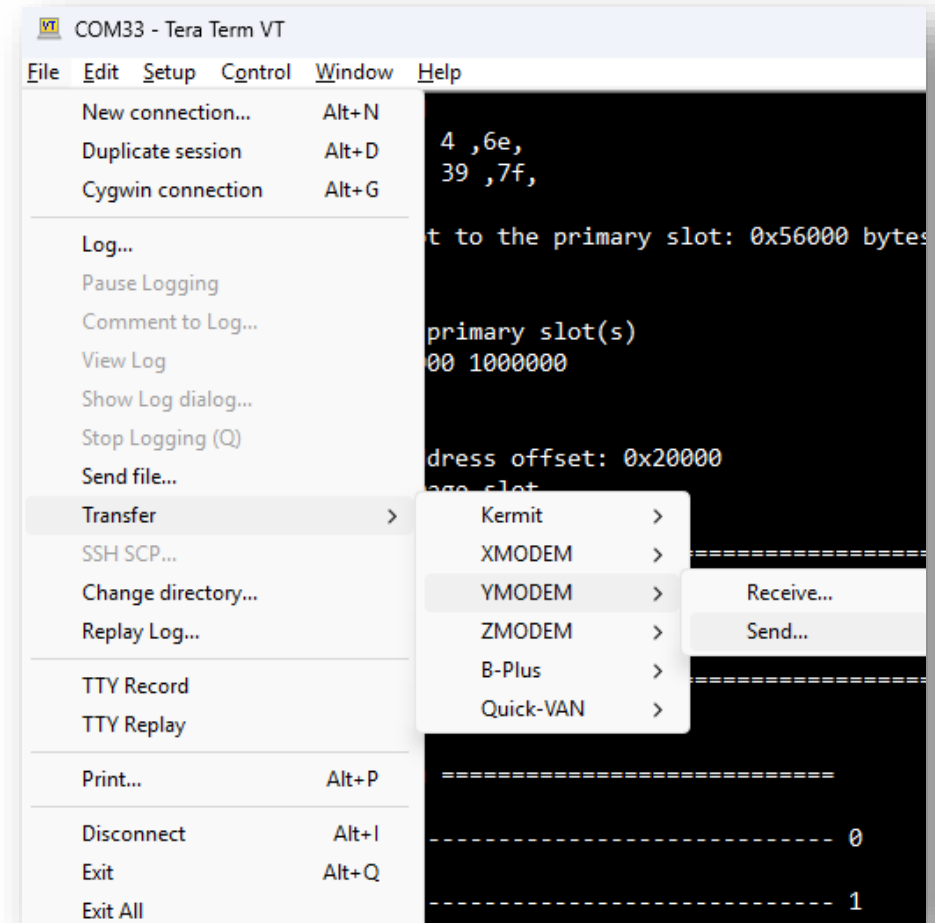
Download App Image ----- 2

Exit from FW update menu ----- x

Download App Image
-- Erasing download area

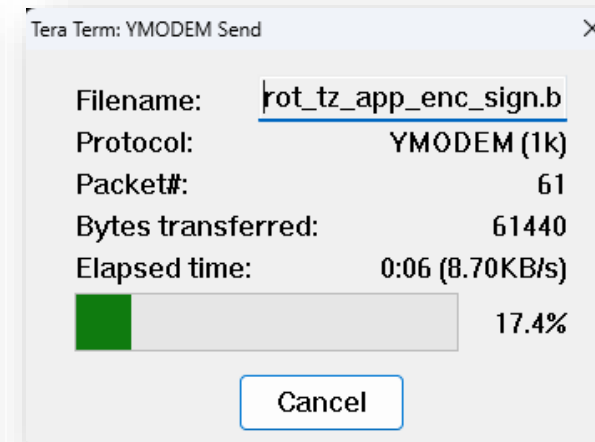
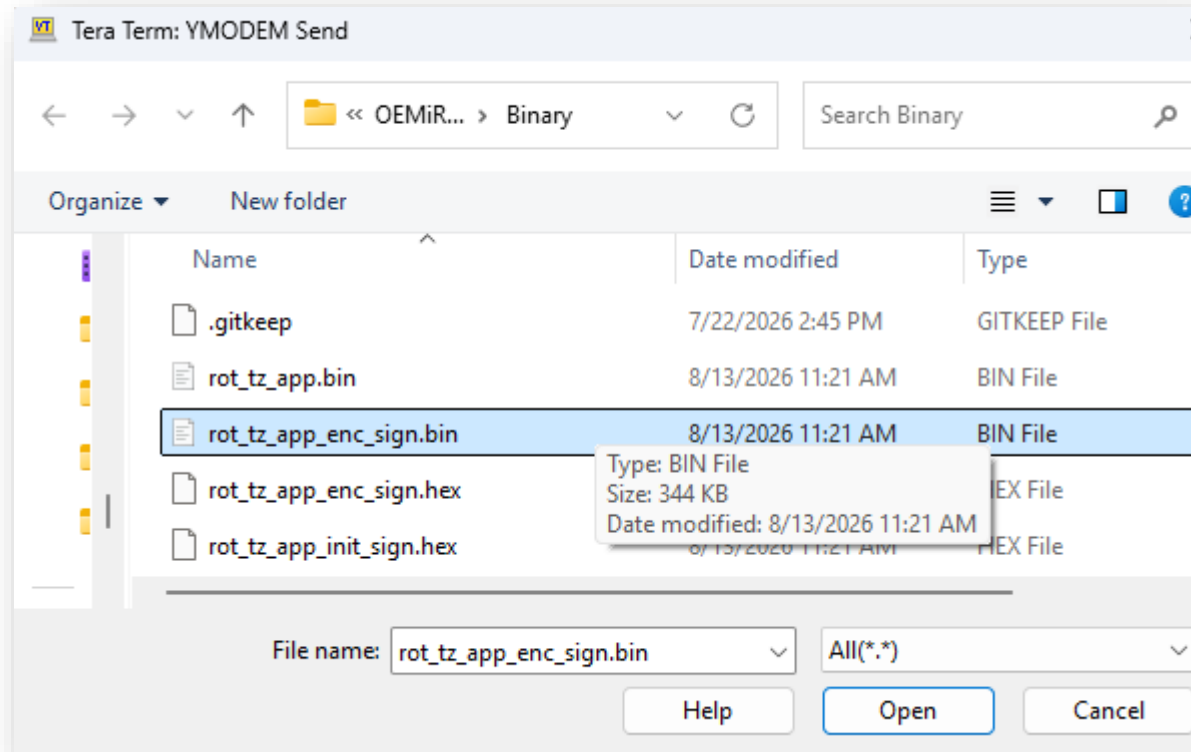
-- Send Firmware

-- -- File> Transfer> YMODEM> Send
```



Run the example: Download App image in User Loder

- Select the encrypted and signed “.bin” file that was generated for FW update
 - Be careful to select the right file in this step (it should be xxx_enc_sign.bin, not xxx_enc_sing.hex)
- For “Primary Only” config, content in primary slot on flash will also be erased first
- Wait for the transmission to complete



```
-- -- File> Transfer> YMODEM> Send
.....e_result = 0 , 3

-- -- Programming Completed Successfully!

-- -- Bytes: 352256

-- App Image correctly downloaded
```

Run the example: Download App image in User Loder

- Select option '1' or reset the board

```
-- -- Programming Completed Successfully!

-- -- Bytes: 352256

-- App Image correctly downloaded

===== New Fw Image =====

Jump to system bootloader ----- b

Reset to trigger Installation ----- 1

Download App Image ----- 2

Exit from FW update menu ----- x

-- Install image : reboot
```

- Then new version app will be installed and run

```
[INF] Starting validation of primary slot(s)
[INF] verify counter 0 1000000 1000000
[INF] counter 0 : ok
[INF] verify sig key id 0
[INF] checking public key 47 5b
[INF] verifying signature hlen 20
[INF] signature OK
[INF] Bootloader chainload address offset: 0x20000
[INF] Jumping to the first image slot
```

```
=====
=                (C) COPYRIGHT 2023 STMicroelectronics                =
=                Built on Aug 13 2026 11:43:21                        =
=                User App #B                                          =
=====
```

```
===== Main Menu =====

Start User Loader ----- 0

Start System BootLoader ----- 1

Selection :
```


Run the example: Start System Bootloader from App

- In version [h553_v1.7.0_oemirot_invt_primaryonly_sysbl.7z](#), it is also possible to start system bootloader from example Application
- Select option '1' in App menu will launch system bootloader

```
=====
=              (C) COPYRIGHT 2023 STMicroelectronics              =
=              Built on Aug 13 2026 11:43:21                      =
=              User App #B                                       =
=====

===== Main Menu =====

Start User Loader ----- 0

Start System BootLoader ----- 1

Selection :

Start config before jumping to the bootloader
Standard Bootloader started
If you want to connect through USART interface, disconnect your TeraTerm
Start download with STM32CubeProgrammer through supported interfaces (USART/SPI/I2C/USB)
```

Our technology starts with You



Find out more at www.st.com

© STMicroelectronics - All rights reserved.

ST logo is a trademark or a registered trademark of STMicroelectronics International NV or its affiliates in the EU and/or other countries.

For additional information about ST trademarks, please refer to www.st.com/trademarks.

All other product or service names are the property of their respective owners.

